

TASK- 1 -CAPABILITY LIST:

Q1. From your Assignment 1 brief, list every distinct job the system does — at least six Capabilities (the verbs).

Capability (Verb)	What It Covers
Register / Authenticate Users	Create new accounts and log in existing users (students, faculty/staff, vendors, admins).
Discover Food	Search and browse vendors, view restaurant menus and individual item details.
Manage Cart	Add, remove, and update items and quantities in a user's cart before ordering.
Place & Manage Orders	Create an order from the cart, accept/reject/cancel it, and update its status.
Process Payments	Authorize, capture, and refund payments for an order.
Track Delivery	Assign a rider/pickup method to an order and track its delivery status.
Manage Vendors & Menus	Create and update vendor profiles, menus, and menu item availability/pricing.
Submit & View Reviews	Let users submit feedback/ratings and let others view reviews for a vendor or item.
Send Notifications	Notify users about order and account events across channels (SMS, email, push).
Administer the Platform	Manage users, vendors, and overall system data as a campus administrator.

TASK - 3 - SERVICE CONTRACTS

Each table lists the operations a service exposes to other services. Internal storage, record formats, and query mechanics are not exposed only the operation, its inputs, its outputs, and the errors it can return.

Boundary rule: a service owns its data. Other services reach that data only by calling the operations below.

Accounts Service

Owns: users, addresses, login | Member: Kaushik Rakshit

Operation	Input	Output	Errors
authenticateUser	Email, password	Session token, user reference, display name	INVALID_CREDENTIALS ACCOUNT_LOCKED USER_NOT_FOUND
getUser	User reference	Name, email, phone, account created date	USER_NOT_FOUND UNAUTHORIZED
getAddress	User reference, address reference (optional — omit for default address)	Label, address line, city, pincode, default flag	USER_NOT_FOUND ADDRESS_NOT_FOUND UNAUTHORIZED

Catalogue Service

Owns: restaurants, menus, prices | Member: Uday Gupta

Operation	Input	Output	Errors
listRestaurants	Optional search text, open-only flag	List of restaurant summaries: name, description, address, phone, open status	INVALID_FILTER
getMenu	Restaurant reference	List of menu items: name, description, availability, current price	RESTAURANT_NOT_FOUND RESTAURANT_CLOSED
checkItem	Menu item reference, quantity	Availability status, current unit price, owning restaurant reference	ITEM_NOT_FOUND ITEM_UNAVAILABLE RESTAURANT_CLOSED

Orders Service

Owns: carts, orders, status | Member: Yash Raj

Operation	Input	Output	Errors
placeOrder	User reference, restaurant reference, list of (menu item reference, quantity), delivery address reference	Order reference, order summary: total amount, status, created time	EMPTY_CART ITEM_UNAVAILABLE RESTAURANT_CLOSED INVALID_ADDRESS PRICE_MISMATCH
getOrder	Order reference, requesting user reference	Order details: items, total amount, status, delivery address, timestamps	ORDER_NOT_FOUND UNAUTHORIZED
cancelOrder	Order reference, requesting user reference, cancellation reason (optional)	Confirmation, updated order status	ORDER_NOT_FOUND UNAUTHORIZED ORDER_NOT_CANCELLABLE

Operation	Input	Output	Errors
updateStatus	Order reference, new status, calling service identity	Confirmation, updated order status	ORDER_NOT_FOUND INVALID_STATUS_TRANSITION UNAUTHORIZED

Payments Service

Owns: transactions, refunds | Member: Mukesh Mourya

Operation	Input	Output	Errors
authorizePayment	Order reference, user reference, amount, payment method	Transaction reference, authorization status	PAYMENT_DECLINED INVALID_AMOUNT INVALID_PAYMENT_METHOD DUPLICATE_REQUEST
capturePayment	Transaction reference	Confirmation, final transaction status	TRANSACTION_NOT_FOUND ALREADY_CAPTURED AUTHORIZATION_EXPIRED
refundPayment	Transaction reference, refund amount, reason	Refund reference, refund status	TRANSACTION_NOT_FOUND TRANSACTION_NOT_CAPTURED REFUND_EXCEEDS_ORIGINAL ALREADY_REFUNDED

Delivery Service

Owns: riders, assignments | Member: Ritesh Jadhav

Operation	Input	Output	Errors
assignRider	Order reference, pickup restaurant reference, delivery address reference	Assignment reference, assigned rider summary: name, vehicle type	NO_RIDERS_AVAILABLE ORDER_ALREADY_ASSIGNED INVALID_ORDER
getAssignment	Order reference or assignment reference	Assignment details: rider summary, status, assigned time, delivered time	ASSIGNMENT_NOT_FOUND
updateDeliveryStatus	Assignment reference, new status, calling identity	Confirmation, updated status	ASSIGNMENT_NOT_FOUND INVALID_STATUS_TRANSITION

Notifications Service

Owns: message log | Member: Mukesh Mourya

Operation	Input	Output	Errors
sendNotification	User reference, type, channel, message content	Message reference, delivery status (queued or sent)	USER_NOT_FOUND INVALID_CHANNEL MESSAGE_TOO_LONG
getMessageStatus	Message reference	Current status (sent, failed, pending), sent time	MESSAGE_NOT_FOUND
logMessage	User reference, type, channel, message content, status	Confirmation, message reference	INVALID_INPUT

TASK – 4- PLACEORDER OPERATION

SPECIFICATION

Inputs

Field	Type	Required	Description
userReference	Reference (ID)	Yes	User placing the order
restaurantReference	Reference (ID)	Yes	Restaurant the order is placed with
items	List of (menuItemReference, quantity)	Yes, min 1	Menu items and quantities ordered
deliveryAddressReference	Reference (ID)	Yes	User's saved address to deliver to

Outputs

Field	Type	Description
orderReference	Reference (ID)	Unique ID for the new order
orderSummary.totalAmount	Currency amount	Computed total price
orderSummary.status	Status label	Initial order status
orderSummary.createdTime	Timestamp	Time the order was created

Error Cases

Error Code	When It Occurs
EMPTY_CART	Items list is empty
ITEM_UNAVAILABLE	One or more items unavailable / out of stock / not on menu
RESTAURANT_CLOSED	Restaurant is currently closed
INVALID_ADDRESS	Address reference invalid or not owned by user
PRICE_MISMATCH	Expected price does not match current menu price

Internal Details Hidden from Callers

Detail	What Is Hidden
Data storage & schema	Orders/carts table structure, keys, indexes, how orderReference is generated, audit/history logs
Price calculation	How Catalogue prices are looked up and summed; caching of menu/price data

Detail	What Is Hidden
Cross-service calls	Sequence/number of calls to Catalogue (checkItem) and Accounts (getAddress); retry/timeout/fallback logic
Stock locking	Internal reservation of menu-item stock to prevent over-selling on concurrent orders
Order state machine	Full set of internal statuses and allowed transitions between them
Idempotency & transactions	Deduplication of retried requests; transaction boundaries for writing the order
Logging & observability	Internal logs, metrics, and traces used for monitoring/debugging

TASK – 6: SERVICE PROPERTY CHECK

1. Accounts Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	—	—
Self-contained	Pass	No change needed — it already contains everything required for account management.	—
Has a contract	Pass	Formalize the request, response, and error formats for clarity.	—
Independent	Pass	No change needed — not heavily dependent on other services.	—
Loosely coupled	Pass	Do not allow other services to directly access the Accounts database; use APIs or events.	Another service is given direct access to the Accounts database instead of going through authenticateUser / getUser / getAddress.

2. Catalogue Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	—	High read load during peak ordering hours (e.g., lunch rush) overwhelms the service if it is not scaled or cached properly.
Self-contained	Pass	No change required.	—
Has a contract	Pass	Formalize the request, response, and error formats.	—
Independent	Pass	No changes are required.	—
Loosely coupled	Pass	Do not allow other services to directly access the Catalogue database.	—

3. Orders Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	—	—

Property	Result	Fix / Action	Could Fail If
Self-contained	Pass	No change required.	—
Has a contract	Pass	Formalize the service contract, including request, response, and error formats.	—
Independent	Pass	No changes required as long as it does not directly access other services' databases.	—
Loosely coupled	Pass	Communication with other services should happen through APIs or events rather than direct database access.	Payments is called synchronously with no timeout or fallback, so a slow Payments Service cascades into failed orders.

4. Payments Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	—	The upstream third-party payment gateway is down or rate-limiting, making authorizePayment/capturePayment effectively unreachable.
Self-contained	Pass	No change required.	—
Has a contract	Pass	No change required, though the contract can be formally documented with request, response, and error formats.	—
Independent	Pass	Manages its own payment and refund data without directly depending on another service's database.	—
Loosely coupled	Pass	Do not allow direct access to the Orders or Accounts databases; use APIs or events.	—

5. Delivery Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	—	The rider location/GPS tracking integration goes down, or the service is not reachable during a spike in delivery requests.
Self-contained	Pass	No change required.	—
Has a contract	Pass	The required operations are defined in the service contract.	—
Property	Result	Fix / Action	Could Fail If
Independent	Pass	Do not allow direct access to other services' databases.	—
Loosely coupled	Pass	Communication with the Orders Service should happen through APIs or events, not through direct database access.	—

6. Notifications Service

Property	Result	Fix / Action	Could Fail If
Reachable	Pass	—	The third-party SMS/email/push provider is down or throttling, so sendNotification cannot actually deliver messages.
Self-contained	Pass	No change required.	—
Has a contract	Pass	The contract defines the required operations, such as sending and tracking notifications.	—
Independent	Pass	Should not directly access the Accounts or Orders databases.	—
Loosely coupled	Pass	Communication should happen through APIs or events rather than direct database access.	—